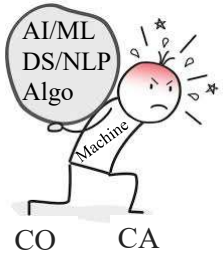
CS-2710-1 Spring 2026

Computer Organization and Systems (COS)



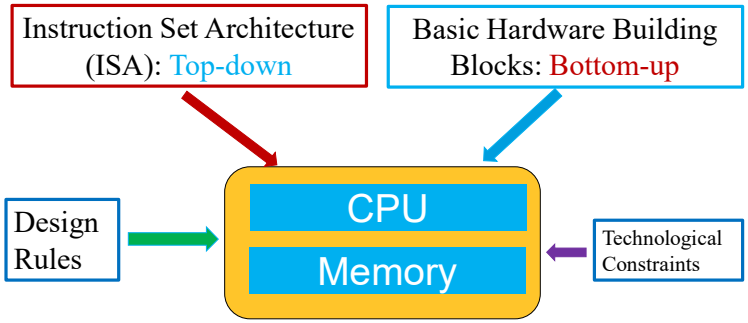
Lecture #13

18 March 2026

Ashoka University

BBB

How to design a processor?

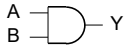


Processor (CPU) Design: Begin from ISA

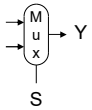
- We have studied CO-fundamentals, MIPS, and ALU design. Now, we're ready to implement the MIPS "core":
  - load-store instructions: `lw, sw`
  - reg-reg instructions: `add, sub, and, or, slt`
  - control flow instructions: `beq, j`

Basic Building Blocks

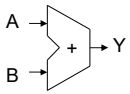
- **AND-gate**
  - **Y = A & B**



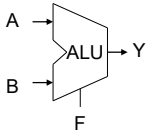
- Multiplexer



- Adder
  - Y = A + B



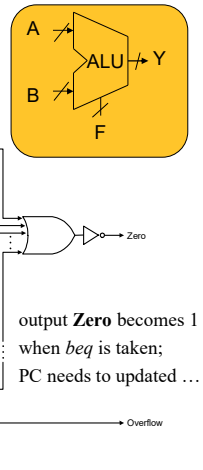
- Arithmetic/Logic Unit
  - Y = F(A, B)

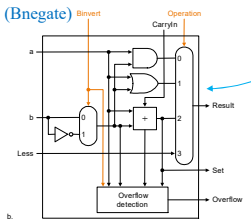


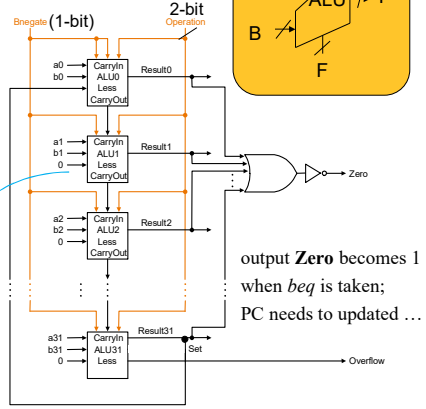
Implementing ALU

Control lines (Bnegate Op1 Op0):

```
000 = and
001 = or
010 = add
110 = subtract
111 = slt
```





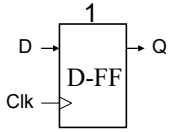


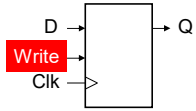
output **Zero** becomes 1 when *beq* is taken; PC needs to updated …

This ALU implements MIPS *add*, *sub*, *and*, *or*, *slt*, and *beq*

Basic Building Blocks

- Register: stores data in a circuit (a chain of D-FFs)
  - Uses a clock signal to determine when to update the stored value
  - Edge-triggered: update when Clk changes from 0 to 1





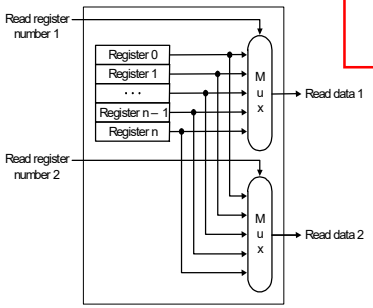
- Register with write control
  - Only updates on clock edge when write control input is 1
  - Used when stored value is required later

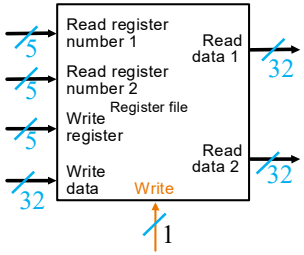
Register File: Read

- Built using D flip-flops

MIPS has a 32 × 32-bit register file
Use for frequently accessed data
Numbered 0 to 31

Assembler names
$t0, $t1, …, $t9 for temporary values
$s0, $s1, …, $s7 for saved variables

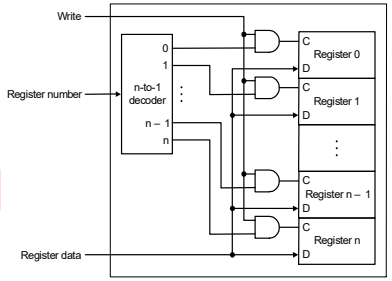




add rd, rs, rt

Register File: Write

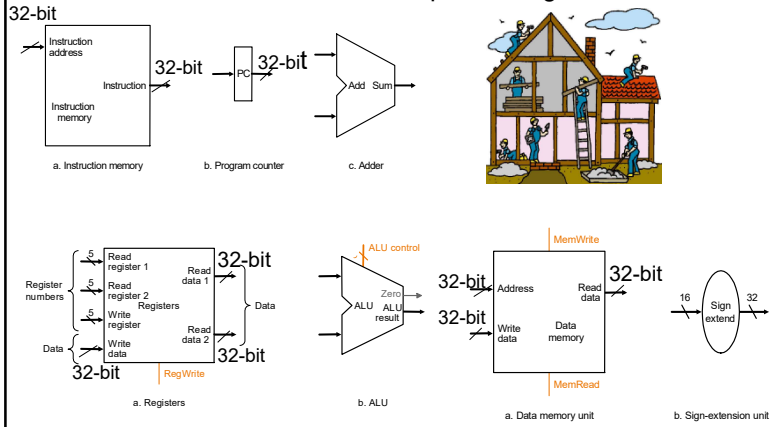
- use the clock to determine when to write, provided *Write* is enabled



add rd, rs, rt

Simple Hardware Implementation of a CPU

- Functional units we need for implementing instructions



MIPS Instruction Execution Cycle

IF → ID/RR → EX → Mem → WB

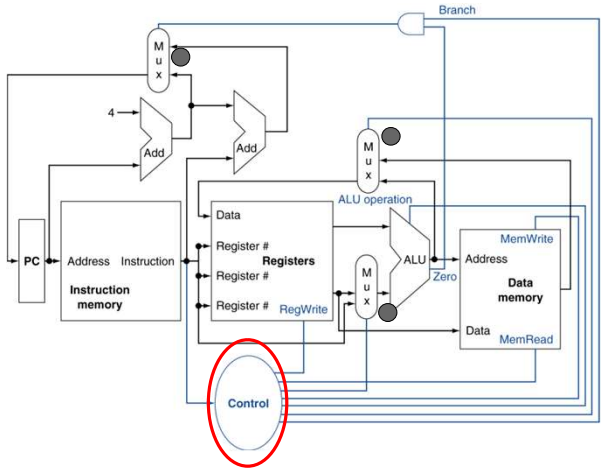
Access memory for load/store;

PC ← target address or PC + 4;

Fetch next instruction from memory address pointed by PC

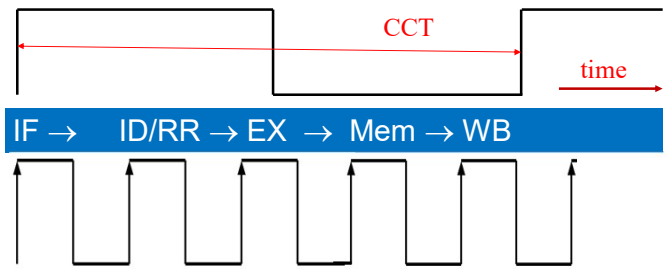
Control

IF → ID/RR → EX → Mem → WB



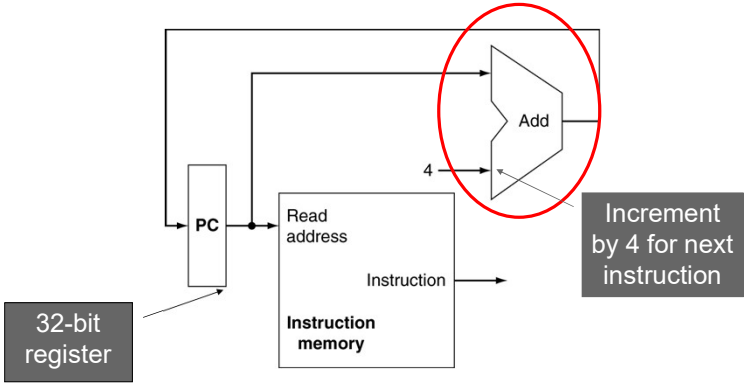
Processor Design

Single-Cycle MIPS Processor



Multi-Cycle Processor (Pipelining)

Instruction Fetch



MIPS Instruction Subset

- **R-type**
  - *add rd, rs, rt*
  - sub, and, or, slt

31–26	25–21	20–16	15–11	10–6	5–0
op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

1. Read registers rs and rt
2. Feed them to ALU
3. Update register file

- **LOAD and STORE**
  - lw rt, rs, imm
  - sw rt, rs, imm

31–26	25–21	20–16	15–0
op	rs	rt	immediate
6 bits	5 bits	5 bits	16 bits

1. Read register rs (and rt for store)
2. Feed rs and immed to ALU
3. Move data between mem and reg

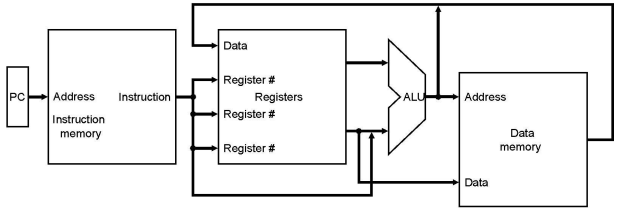
- **BRANCH:**
  - beq rs, rt, imm

31–26	25–21	20–16	15–0
op	rs	rt	displacement
6 bits	5 bits	5 bits	16 bits

1. Read registers rs and rt
2. Feed to ALU to compare
3. Add PC to disp; update PC

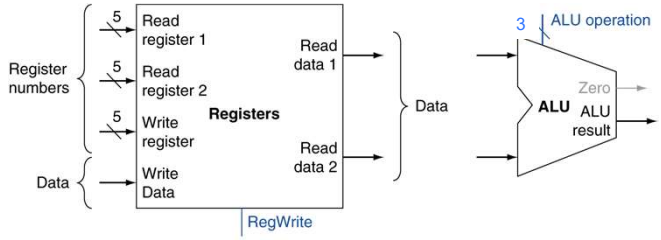
Processor Design

- **Generic Implementation:**
  - all instructions read some registers
  - all instructions use the ALU after reading registers
  - memory accessed & registers updated after ALU
- **Suggests basic design:**



R-Format Instructions

- Read two register operands
- Perform arithmetic/logical operation
- Write register result

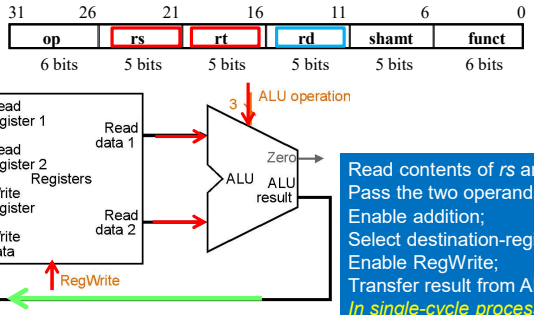


a. Registers

b. ALU

Datapath for Reg-Reg Operations

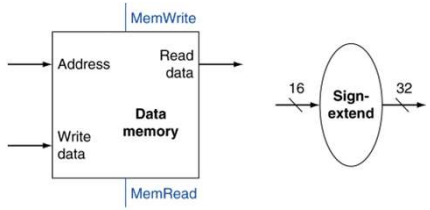
- R[rd] <- R[rs] op R[rt] Example: *add rd, rs, rt*
  - ID's of three registers obtained from *rs, rt,* and *rd* fields of the instruction (two sources and one destination)
  - ALU-operation signal depends on *op* and *funct*



Read contents of *rs* and *rt;*
Pass the two operands to ALU;
Enable addition;
Select destination-register *rd;*
Enable RegWrite;
Transfer result from ALU to *rd;*
In single-cycle processor implementation, all actions are completed in one clock cycle;

Load/Store Instructions

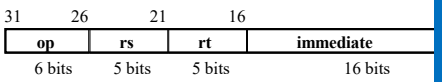
- Read register operands
- Calculate address using 16-bit offset
  - Use ALU, but sign-extend offset
- Load: Read memory and update register
- Store: Write register value to memory



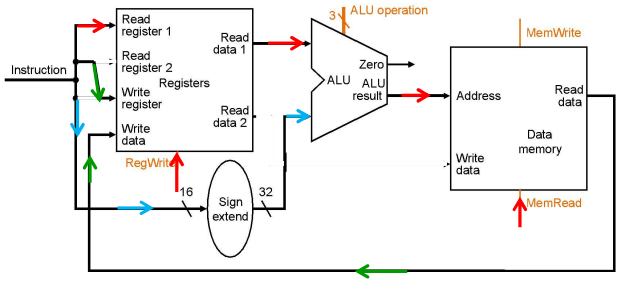
a. Data memory unit b. Sign extension unit

Datapath for Load Operations

R[rt] <- Mem[R[rs] + SignExt[imm16]] Exa

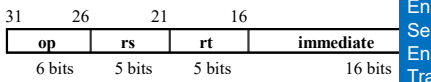


Read contents of *rs*
Pass (*rs*) and sign-ext (imm) to ALU;
Enable addition to compute address;
Select dest register *rt*
Enable MemRead
Read memory location pointed by the computed address;
Transfer data from Mem to *rt;*

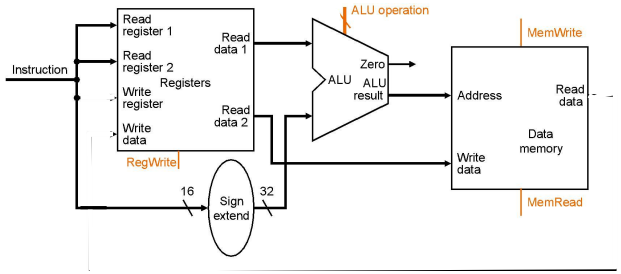


Datapath for Store Operations

Mem[R[rs] + SignExt[imm16]] <- R[rt

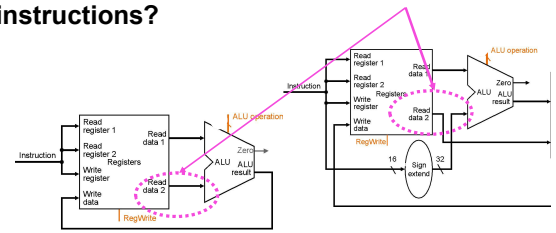


Read contents of *rs*
Pass (*rs*) and sign-ext (imm) to ALU;
Enable addition to compute address;
Select and read dest register *rt*
Enable MemWrite
Transfer (*rt*) to Mem location pointed by the computed address



Combining datapaths

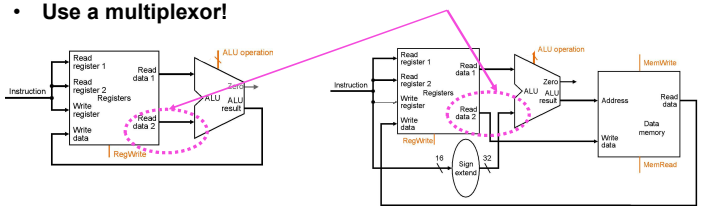
- **How do we allow different datapaths for different instructions?**



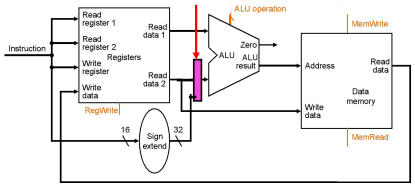
R-type Store

Combining datapaths

- **How do we allow different datapaths for different instructions?**
- **Use a multiplexor!**



ALUscr (ALU second-input control)



Branch Instructions

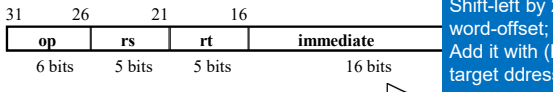
- Read register operands
- Compare operands
  - Use ALU, subtract and check Zero output
- Calculate target address
  - Sign-extend displacement
  - Shift left 2 places (word displacement)
  - Add to PC + 4
    - Already calculated by instruction fetch

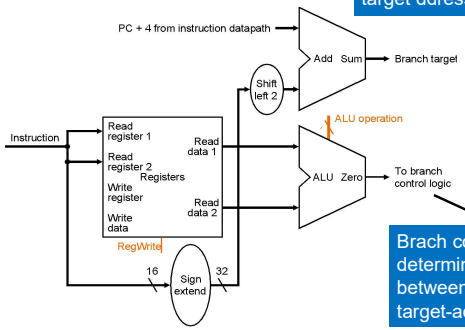
Datapath for Branch Operations

beq rs, rt, imm16

We need to

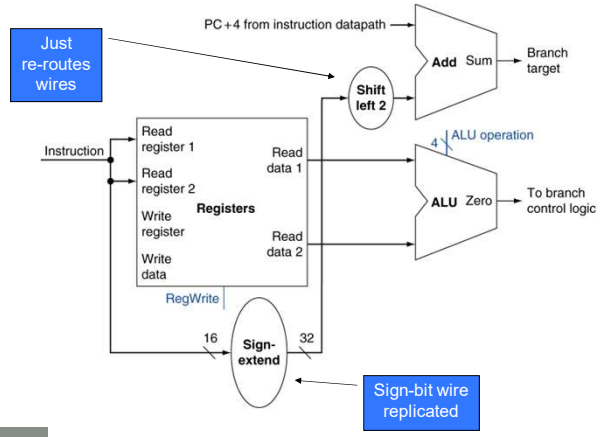
Read contents of *rs* and *rt;*
Compare them in ALU, by enabling ALU-op;
Read *imm* and sign-extend;
Shift-left by 2-bits to compute word-offset;
Add it with (PC+4) to compute target ddress;





Brach control logic determines the choice between two possible target-addresses

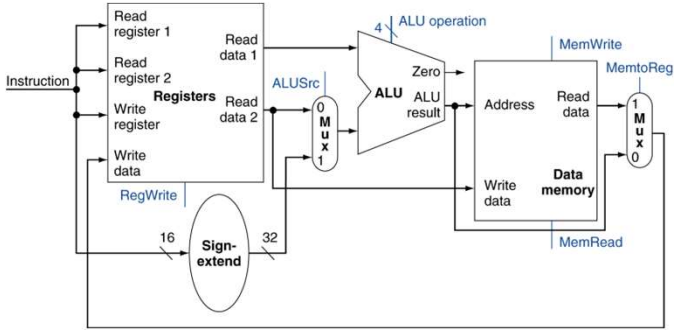
Branch Instructions



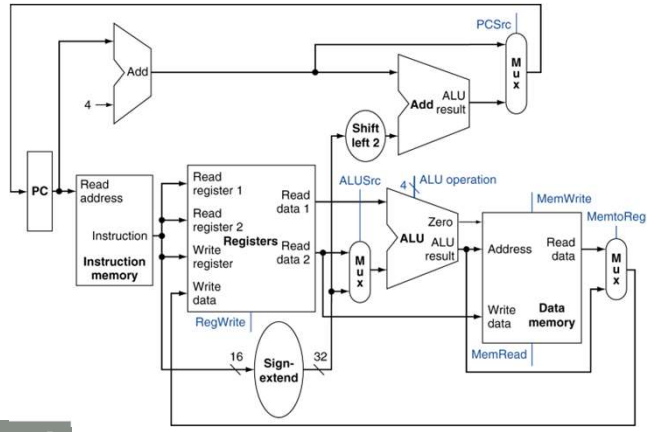
Composing the Elements

- First-cut data path does an instruction in one clock cycle
  - Each datapath element can only do one function at a time
  - Hence, we need separate instruction and data memories
- Use multiplexers where alternate data sources are used for different instructions

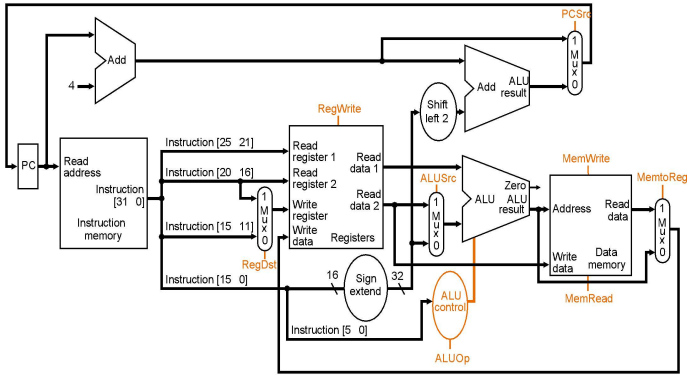
R-Type/Load/Store Datapath



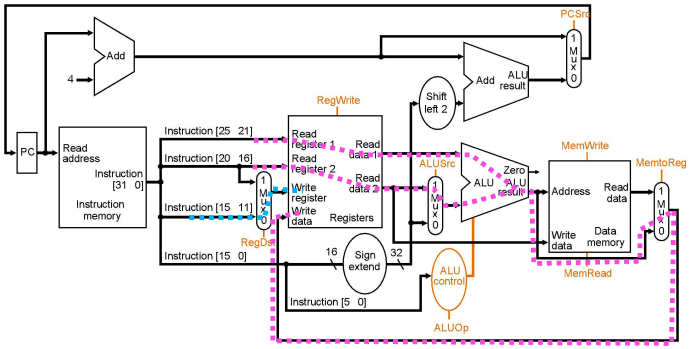
Full Datapath



All together: the single cycle datapath

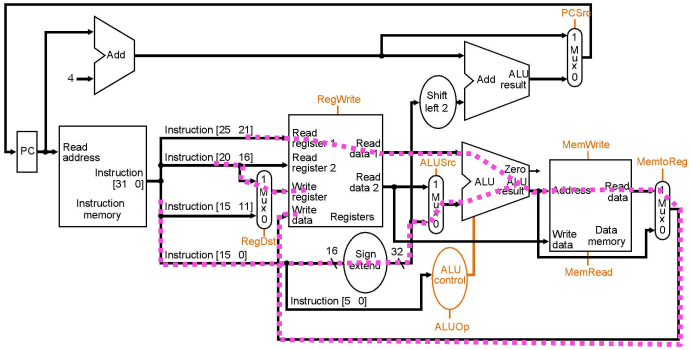


The R-Format (e.g. *add*) Datapath



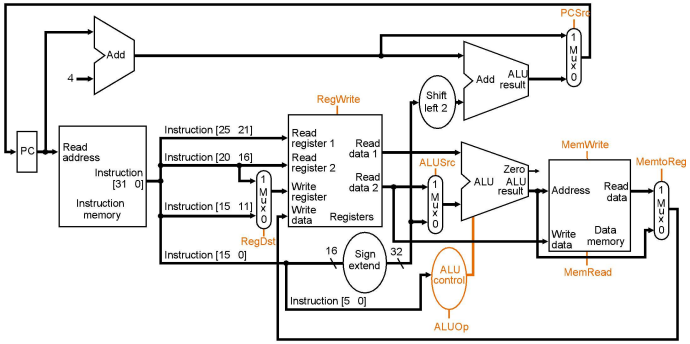
Need Control Signals: ALUsrc = 1, ALUop ="add", MemRead = 0, MemWrite = 0, MemToReg = 0, RegDst = 0, RegWrite = 1, and PCsrc = 1.

The Load Datapath

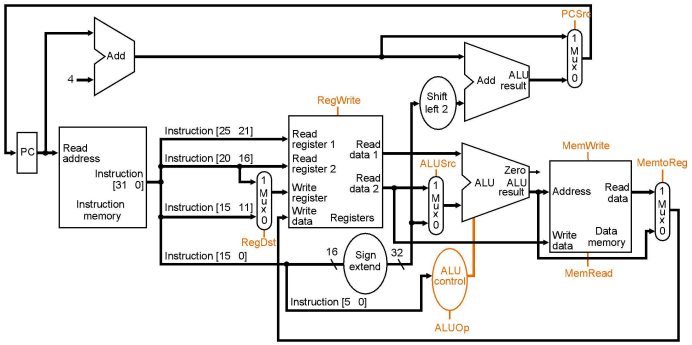


What control signals do we need for load??

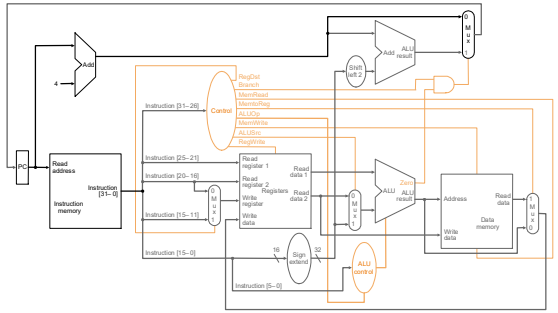
The Store Datapath



The beq Datapath



Control Signal Requirement



Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch	ALUOp1	ALUp0
R-format	1	0	0	1	0	0	0	1	0
`lw`	0	1	1	1	1	0	0	0	0
`sw`	X	1	X	0	0	1	0	0	0
`beq`	X	0	X	0	0	0	1	0	1